

**Lab 12: APR Flow with Innovus –
Powerplan and Placement (1%)**

Assigned on Nov. 20, 2025

Due day in **Nov. 27, 2025**

Objective

In this lab, you will learn how to use Innovus to perform a basic APR flow

Demo checklist:

- ☐ Powerplan
- ☐ Placement result and corresponding IR drop map
- ☐ Timing report after placement
- ☐ Questions:
 1. What is the function of standard cell rails?
 2. Why should we use tie cells to connect constant pins? Can we directly connect constant pins to power/ground ports?
 3. What is IR drop?
 4. Why do we include waveform for power analysis?

Action Items

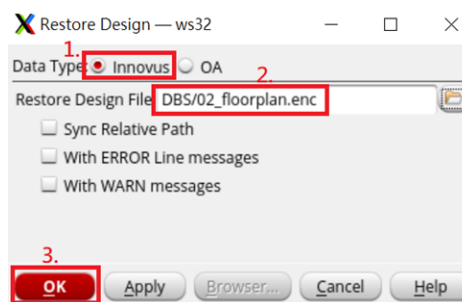
I. Powerplan

1. In the **lab11/innovus/run**, open the GUI of Innovus by:

\$ innovus

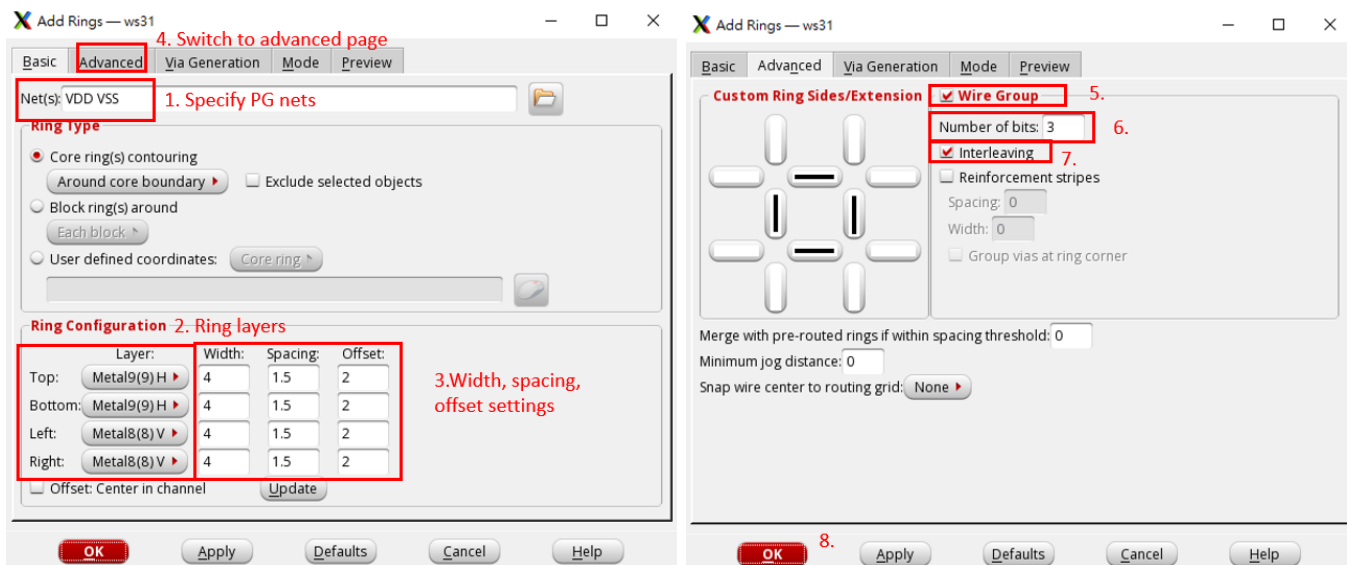
2. Open design from the previous step.

“File->Restore Design...”



3. Create core ring

“Power->Power Planning-> Add Ring...”

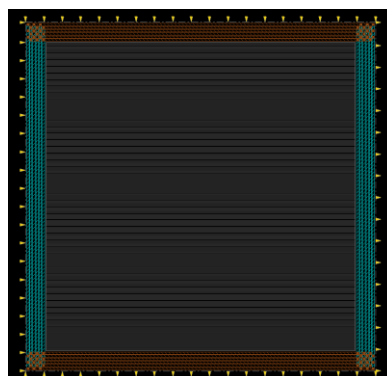


Number of bits: Specifies how many sets of VDD/VSS pair are created
Interleaving: Alternate the VDD and VSS in wire groups.

Note: The spacing is chosen based on “DRC: Metal spacing rules”

Note: The ring width is chosen based on the power consumption of the chip.

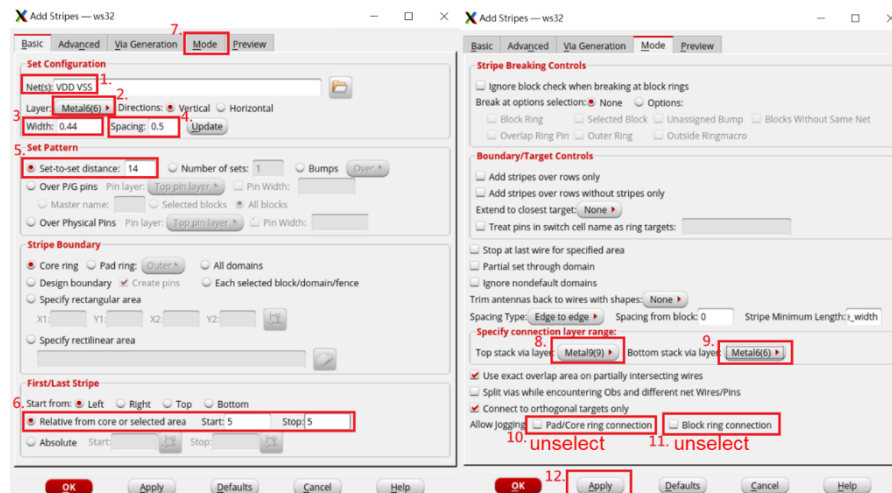
Note: Due to “DRC: Metal width rules,” ring width cannot be too large. Thus, we use multiple pairs of power rings to provide enough power supply for your design.



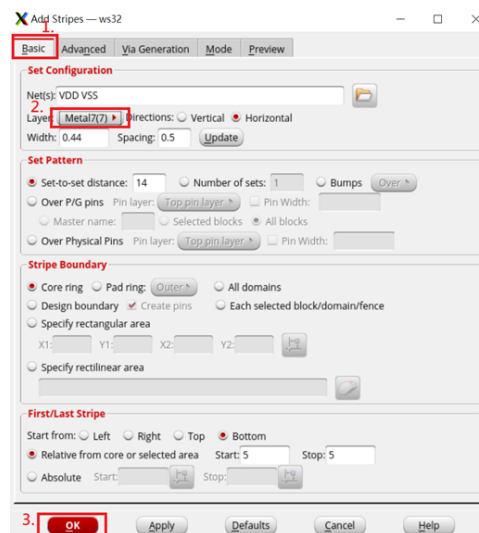
4. Create power stripe

“Power->Power Planning-> Add Stripes...”

(a.) Add vertical stripes



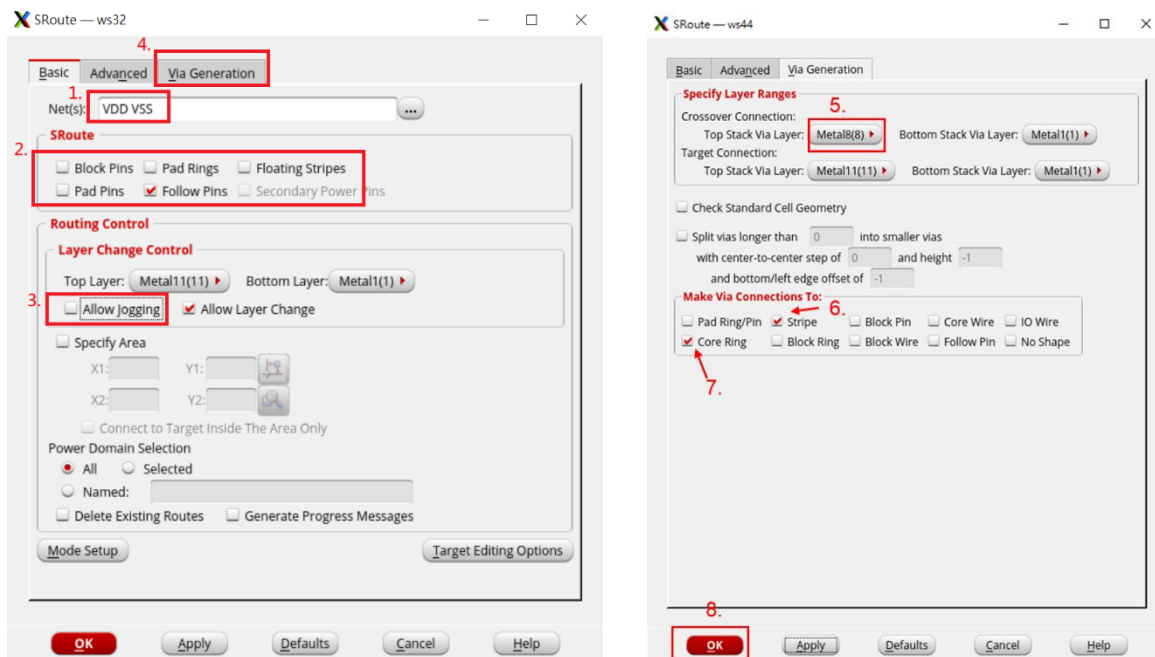
(b.) Add horizontal stripes



Note: As mentioned in the lecture, standard cells are surrounded by **Power/Ground rail (Metal 1)** segments on top and bottom. For routing complexity concern, the direction of each metal layer in the APR flow will follow the rail direction of the standard cells. In our case, we use **odd** layers (Metal 1, Metal 3, ..., Metal 9) for **horizontal** metals and **even** layers (Metal 2, Metal 4, ..., Metal 8) for **vertical** metals. Though this is not a strict rule, violating the metal direction will introduce routing difficulty (more routing resources are required).

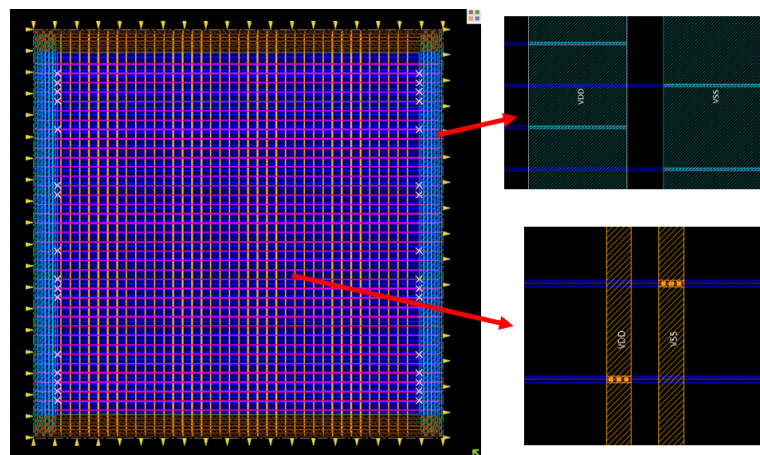
5. Connect Follow Pin

“Route->Special Route...”

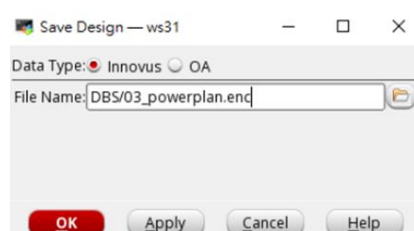


Some Xs show up because rails are blocked by stripe via, while our power mesh is dense enough for affordable IR drop, press “**shift+v**” to neglect it.

The power rails have been created. Please check if the rails have been connected to power stripes and core rings correctly.



6. Save your design



II. Placement

7. Create Path Groups

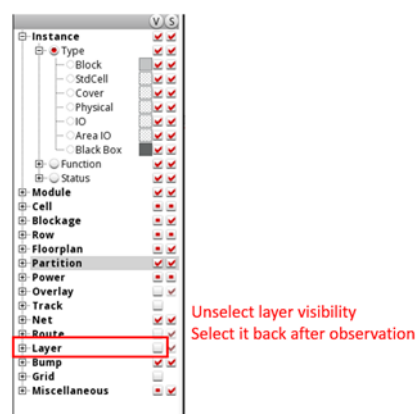
```
$ innovus > createBasicPathGroups -expanded
```

```
$ innovus > get_path_groups
```

8. Add Tap cells

```
$ innovus > addWellTap -cell FILL4 -cellInterval 20 -checkerBoard
```

Where are the Tap cells placed?



9. Run placement

Note: This step might take about 10 minutes.

```
$ innovus > setPlaceMode -reset
```

```
$ innovus > place_opt_design
```

10. Make sure there are no violating paths

optDesign Final Summary							
Setup views included: AV_max							
Setup mode	all	reg2reg	reg2cgate	in2reg	reg2out	in2out	default
WNS (ns):	2.089	2.089	4.463	7.882	8.530	N/A	0.000
TNS (ns):	0.000	0.000	0.000	0.000	0.000	N/A	0.000
Violating Paths:	0	0	0	0	0	N/A	0
All Paths:	22140	19877	2187	43	33	N/A	0
DRVs	Real		Total				
	Nr nets(terms)	Worst Vio	Nr nets(terms)				
max_cap	0 (0)	0.000	0 (0)				
max_tran	6 (12)	-0.188	6 (12)				
max_fanout	602 (602)	-45	603 (603)				
max_length	0 (0)	0	0 (0)				

11. (Optional) Optimize Design

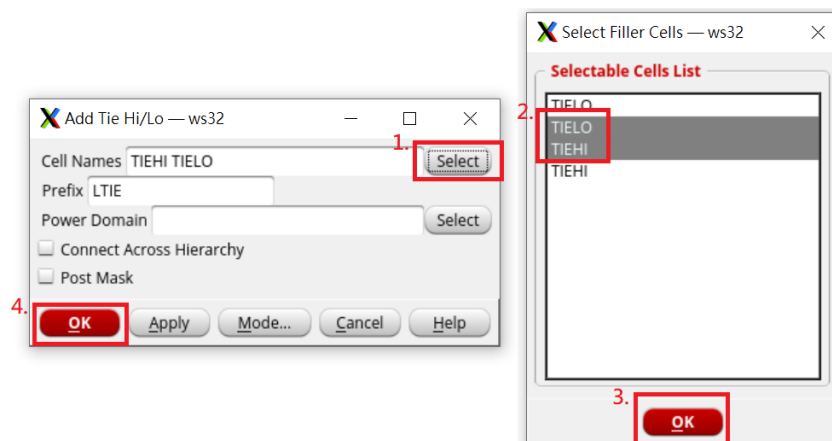
If timing does not pass during above steps.

“ECO->Optimize Design...”



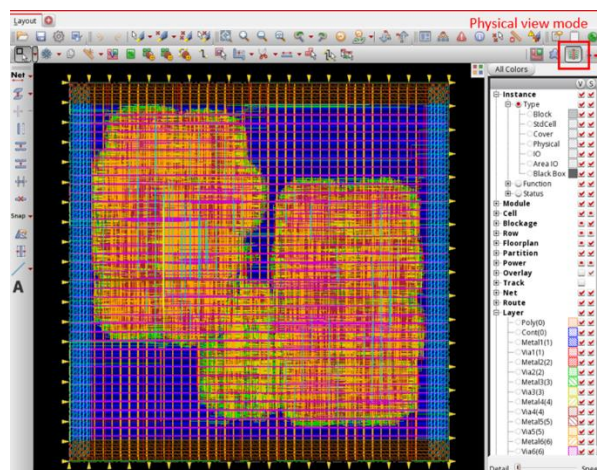
12. Add Tie High/Low Cells

“Place->Tie HI/LO cell->Add...”



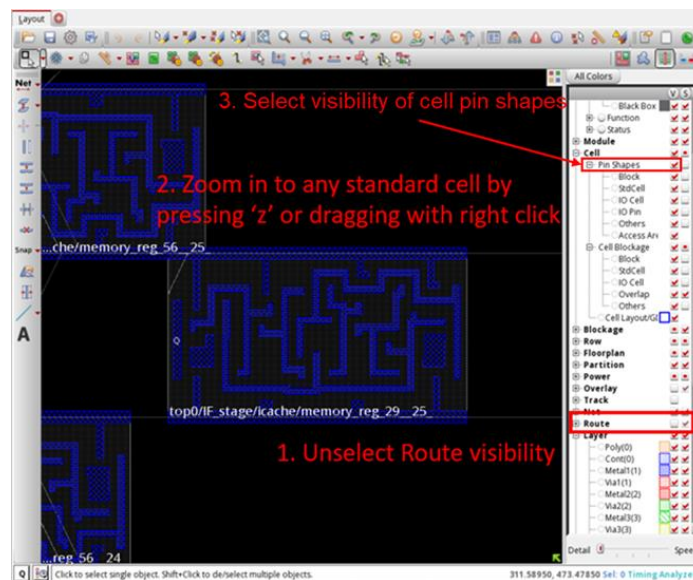
Note: There are redundant options in the GUI window. Just arbitrarily select the ones we need.

13. Switch to physical view mode to see whether the standard cells have been placed on the layout.



14. Observation

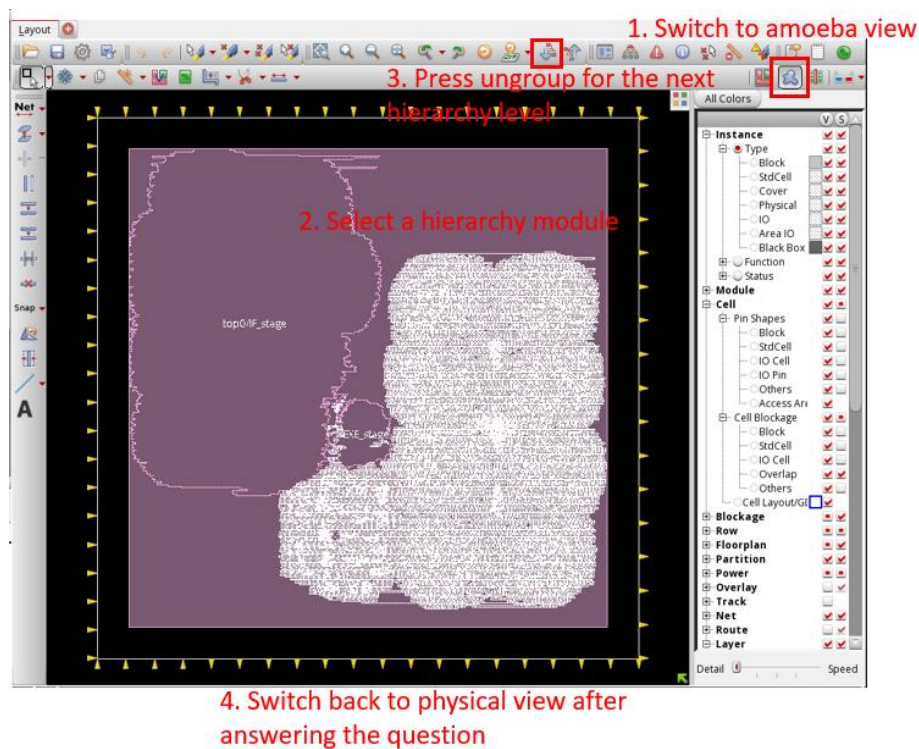
(a) Show FRAM view of standard cells



Q: How do standard cells connect with power/ground pins?

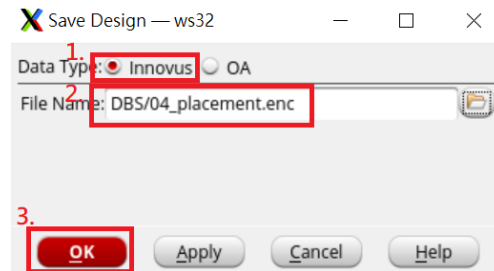
(b) Show the hierarchy groups

Q: How does the layout partitioned?



15. Save design

“File->Save Design...”



III. Static Power Analysis and Early Rail Analysis

Early rail analysis could be done at **floorplan stage, after placement, and post-route** to analyze power-grid integrity.

1. Static Power Analysis

Static power analysis estimates the power consumption with the toggle rate of the signals. Although it can be conducted without waveform, Innovus allows us to obtain a more accurate toggling rate from it.

(a) Generate waveform

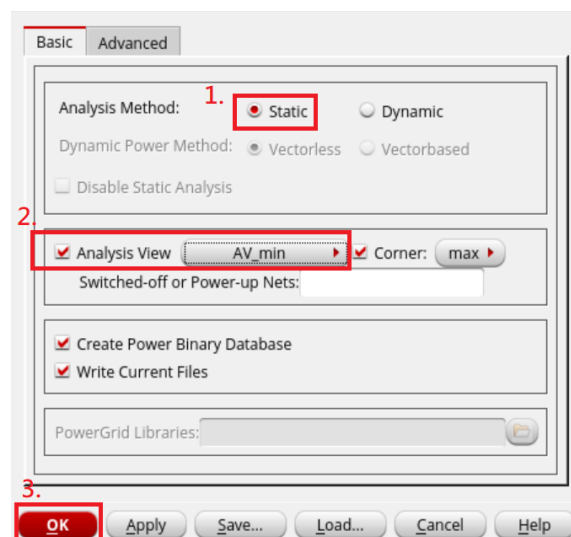
Open another terminal

```
$ cd lab12/sim/
```

```
$ vcs -R -full64 +v2k -f gatesim.f -debug_access+all
```

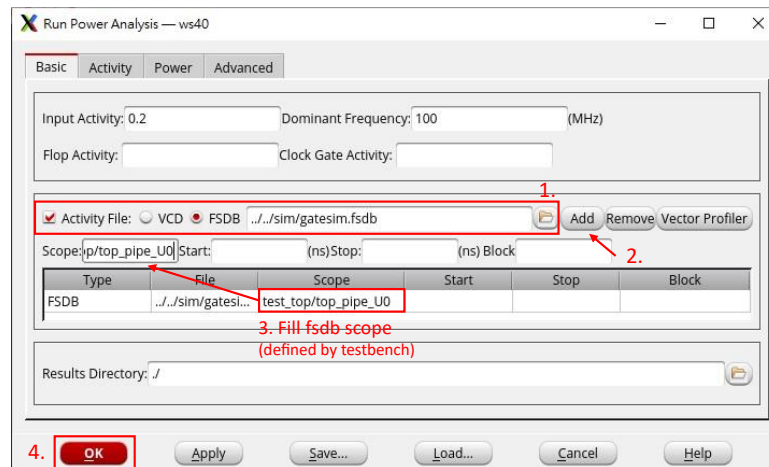
Setup power analysis

“Power->Power Analysis->Setup...”



(b) Run power analysis

“Power->Power Analysis->Run...”



(c) Check the log in the terminal to find the power analysis result

Total internal power = _____ (mW)

Total switching power = _____ (mW)

Total leakage power = _____ (mW)

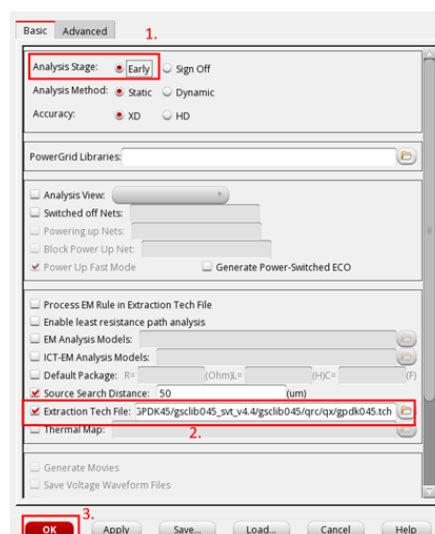
Total power = _____ (mW)

2. Rail Analysis

(a) Setup Rail analysis

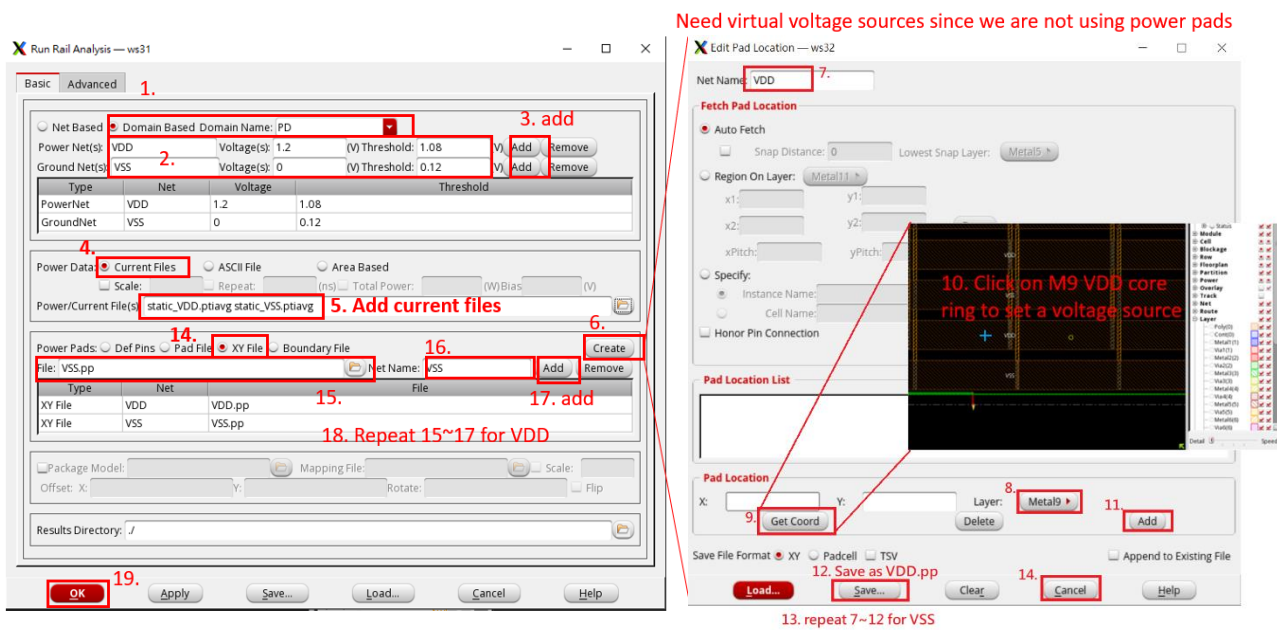
“Power->Rail Analysis->Setup Rail Analysis...”

Extraction Tech file	/usr/cadtool/GPDK45/gsclib045_svt_v4.4/gsclib045/qrc/qx/gpdk045.tch
----------------------	---



(b) Run rail analysis

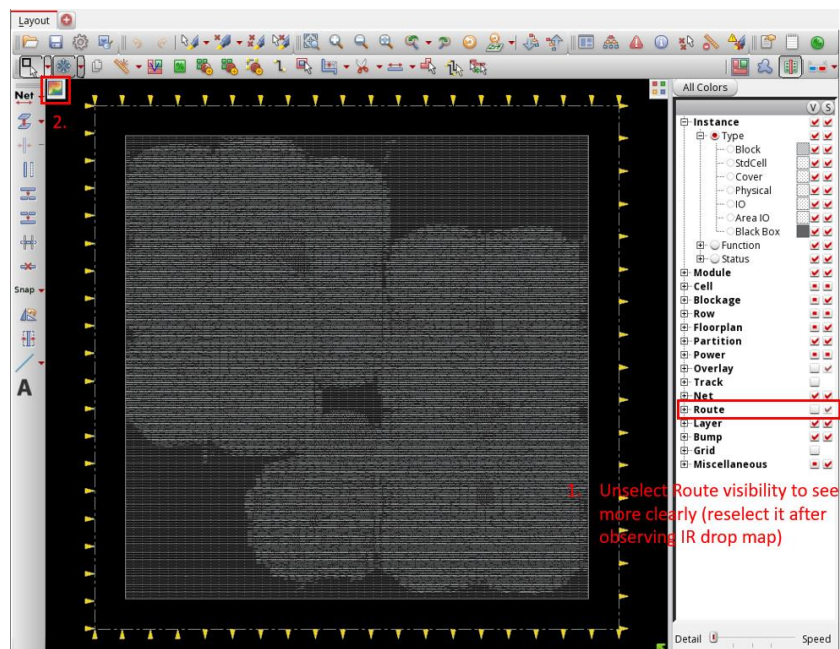
“Power->Rail Analysis->Run Rail Analysis...”

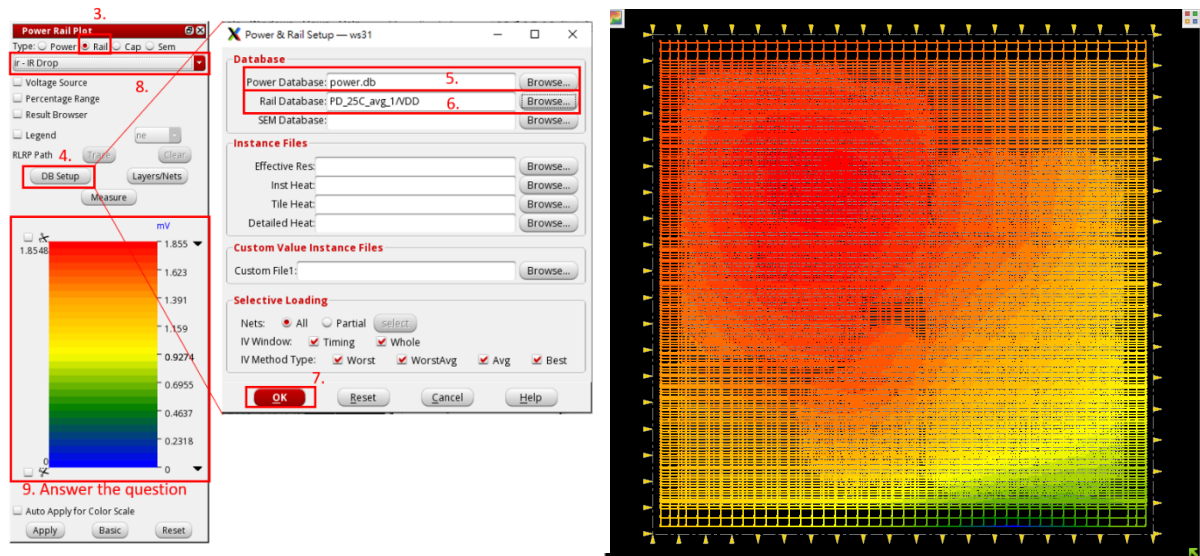


(c) Check the log in the terminal to find the IR drop

(d) Visualize IR drop map

“Power->Report->Power Rail Result”





Q: How big is the worst IR drop? Is that acceptable?

Appendix

Here is the file list of the lab package. Please check if anything missing!

Directory	Filename	Description
innovus/pre_layout/	top_pipe_syn.v	Synthesized netlist
	top_pipe_syn.sdc	Modified SDC for Innovus
	ccopt.cmd	CTS commands
	mmmc.view	MMMC setting file
sim	instruction.txt	CPU simulation pattern
	run_presim.sh	Shell script for pre-simulation
	presim.f	File list for pre-simulation
	run_gatesim.sh	Shell script for gate-level simulation
	gatesim.f	File list for gate-level simulation
	test_top.v	Testbench for the CPU
hdl	top.v	Top module for this CPU
	top_pipe.v	Pipelined design for this CPU
	ID_stage.v	RTL for ID stage
	IF_stage.v	RTL for IF stage
	controller.v	RTL for control signal
	regfile.v	RTL for register file
	ID_EXE.v	RTL for ID to EXE flip flop
	EXE_stage.v	RTL for EXE stage
	alu.v	RTL for alu module

	CPU_define.v	Definition file
	dsram.v	RTL for memory
	PC.v	RTL for program counter
	IF_ID.v	RTL for IF to ID flip flop
syn	.synopsys_dc.setup	DC setup file
	*.tcl	Synthesis scripts
	*.log	Log files
	run_dc.sh	Shell script for synthesis
syn/report	*.out	Report files
	top_pipe_syn.ddc	Netlist & constraints can be read by DC
	top_pipe_syn.saif	Switching activity interchange format (for power)
	top_pipe_syn.sdc	Synopsys design constraints format
	top_pipe_syn.sdf	Standard delay format
	top_pipe_syn.v	Netlist in Verilog
primitime/pre_layout	pt_px.tcl	Script for pre-layout power analysis
primitime/postsim_wa veform	pt_px.tcl	Script for post-layout power analysis with post-sim waveform
primitime/presim_wa veform	pt_px.tcl	Script for post-layout power analysis with pre-sim waveform
lec/pre_layout_lec	run_lec.sh	Shell script for LEC
	*.do	Scripts for RTL-to-netlist LEC
lec/post_layout_lec	run_lec.sh	Shell script for LEC
	*.do	Scripts for netlist-to-post-layout LEC